Trivy

The Complete Practical Guide

■ **Praveen Kankatala**

A Quick Word Before You Start

Security tooling has a reputation for being heavy, slow, and intimidating. Trivy is the opposite. It is one of those rare tools that you can install in a minute, run with a single command, and immediately get something useful back.

This guide is written for engineers who want to actually use Trivy, not just admire its feature list. Each section starts with the why before getting into the how, and the examples are the kind you can copy and paste into a real project. If you are brand new to container security, start at the beginning and work through. If you already know Trivy and are looking for a specific topic, Kubernetes, air-gapped installs, compliance, ignoring noisy CVEs, feel free to jump straight to that chapter.

One last thing: security is never finished. The goal is not to get to zero findings on day one. The goal is to make scanning a normal part of how you build and ship software, so issues are caught when they are cheap to fix instead of expensive.

- **Praveen Kankatala**

1. What Is Trivy?

Trivy is a free, open-source security scanner created by Aqua Security. Think of it as a flashlight you can shine on almost anything in your stack a Docker image, a Git repository, a Terraform folder, a Kubernetes cluster, even an AWS account and have it tell you what looks broken.

Under the hood, it reads what is inside the thing you point it at, compares that against a giant catalogue of known security problems, and produces a report you can act on.

What Trivy Can Find

- Vulnerabilities (CVEs) in operating system packages Ubuntu, Alpine, Debian, Red Hat, and more.
- Vulnerabilities in application libraries used by Python, Node.js, Java, Go, Ruby, .NET, and other ecosystems.
- Misconfigurations in Infrastructure-as-Code files such as Terraform, Kubernetes manifests, CloudFormation, and Docker files.
- Secrets that should not be in source code AWS keys, GitHub tokens, passwords, private keys, and API credentials.
- License issues for open-source components, which matters for compliance and legal review.
- Compliance gaps against industry benchmarks like CIS, NSA, and PCI-DSS.

What is CIS, NSA, and PCI-DSS.

- The **Center for Internet Security (CIS)** is a non-profit organization that identifies, develops, and promotes consensus-based best practices for cyber defense.
- The **NSA (National Security Agency)** is a U.S. government intelligence agency under the Department of Defense. In cybersecurity, organizations typically refer to the joint **NSA-CISA Kubernetes Hardening Guidance** (co-authored with the Cybersecurity and Infrastructure Security Agency).
- The **Payment Card Industry Data Security Standard (PCI-DSS)** is a mandatory, global information security standard governed by the PCI Security Standards Council (founded by Visa, Mastercard, American Express, Discover, and JCB).

Why Teams Choose Trivy

There are plenty of scanners out there. Trivy keeps winning fans for a few simple reasons:

- It runs as a single binary. No account, no login, no paid plan.
- Scans are fast because the database is downloaded once and cached locally.

- It does a lot of jobs reasonably well, which means one tool instead of five.
- The community is large and active, and Aqua Security updates the database constantly.

2. Installation

Always install Trivy from the official sources. Tutorial repositories and personal forks may be outdated or carry unpatched bugs. The official site keeps you on a supported, long-term version.

Linux (Ubuntu / Debian)

Install the prerequisites, add the official Aqua Security signing key, register the repository, then install the package.

```
sudo apt-get install -y wget gnupg

wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key \
  | sudo gpg --dearmor -o /usr/share/keyrings/trivy.gpg

echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] \
  https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -sc) main" \
  | sudo tee /etc/apt/sources.list.d/trivy.list

sudo apt-get update
sudo apt-get install -y trivy
```

macOS

Homebrew is the cleanest path on a Mac.

```
brew install trivy
```

Windows

Use Chocolatey or download the binary from the Trivy releases page on GitHub.

```
choco install trivy
```

Verify the Installation

```
trivy --version
```

3. How Trivy Works

You do not need to know the internals to be productive with Trivy, but five minutes of background here will save you a lot of confusion later. Especially when you start running into things like database downloads, cache directories, or server mode.

The Four Moving Parts

- **Artifact Analyzer:** the first step. It inspects whatever you pointed Trivy at, detects the operating system, and lists every installed package and dependency.
- **Vulnerability Database:** a compact local cache of known security problems with CVE IDs, severity ratings, affected versions, and fixed versions.
- **Matching Engine:** the brain. It compares what the Analyzer found against the database and produces the list of findings.
- **Report Formatter:** the last step. It turns findings into whatever output you asked for a terminal table, JSON, SARIF, HTML, JUnit, or something else.

Where the Data Comes From

Aqua Security runs a backend pipeline that pulls in vulnerability information from many feeds, deduplicates and normalizes it, and packages the result into a small bundle Trivy can grab quickly. The main inputs are:

- The National Vulnerability Database (NVD), the US government's official CVE catalogue.
- Vendor advisories from Debian, Red Hat, Ubuntu, Alpine, SUSE, and other distributions.
- GitHub Security Advisories, which capture community-reported issues.
- Package ecosystem feeds for npm, pip, RubyGems, Go modules, Maven, NuGet, and others.

How Severity Is Decided (CVSS)

Trivy uses the Common Vulnerability Scoring System (CVSS) to rate how dangerous each finding is. The score takes into account how hard it is to exploit, whether user interaction is needed, what privileges are required, and how badly it affects confidentiality, integrity, and availability.

Severity	CVSS Score Range
Low	0.0 – 3.9
Medium	4.0 – 6.9
High	7.0 – 8.9
Critical	9.0 – 10.0

In practice, most teams focus on High and Critical first. Those are the ones most likely to be exploited and most likely to cause real damage if they are.

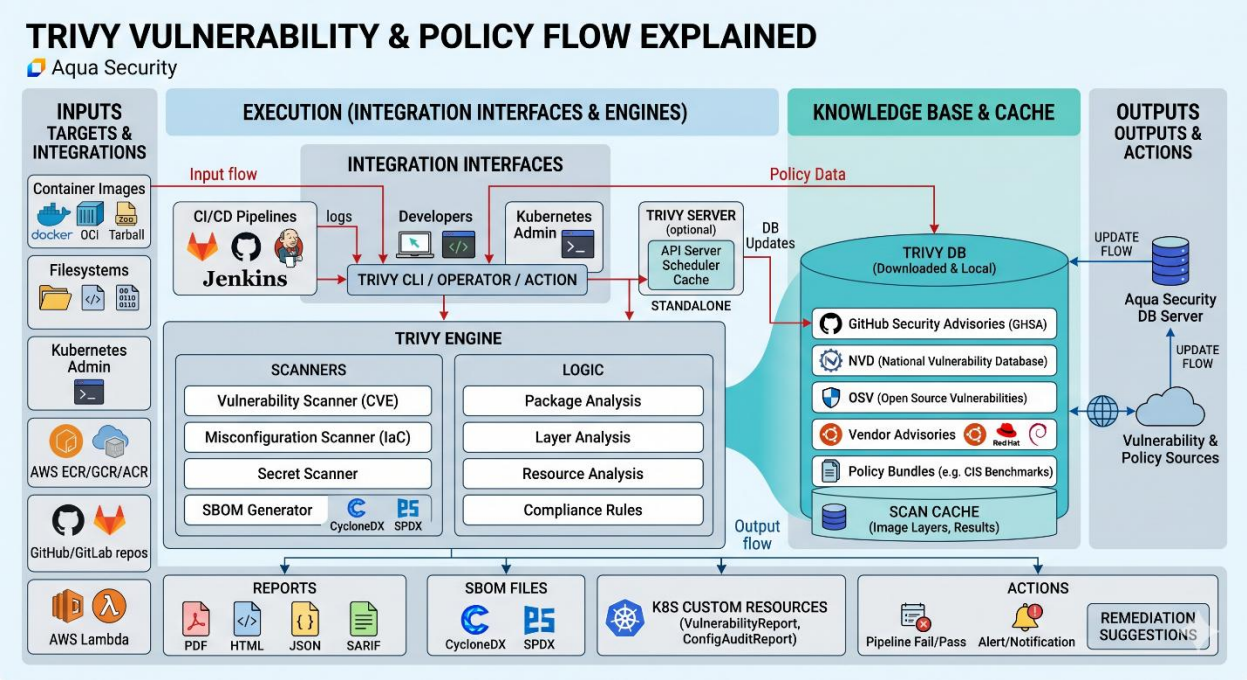
4. High-Level Architecture

Every scan flows from left to right and top to bottom through the layers below. The first table summarises each layer; the rest of the document walks through them in sequence.

Layer	Role in the scan flow
Targets & Integrations	Container images (Docker, OCI, tarball), local filesystems, Kubernetes clusters, IaC files, and cloud accounts that Trivy is asked to inspect.
Integration Interfaces	Entry points that invoke Trivy: CI/CD pipelines (GitHub Actions, GitLab CI, Jenkins), developer machines, and the trivy-operator inside Kubernetes.
Trivy CLI / Operator / Action	The orchestrator. Parses commands, selects the right input handler, drives the engine, and writes results. The same binary powers the CLI, the K8s operator, and the GitHub Action.
Trivy Server (optional)	Client-server mode. A central Trivy Server holds the DB cache; many lightweight clients call it over HTTP. Useful in air-gapped or large-scale CI setups to avoid every runner downloading the DB.
Trivy Engine — Scanners	Pluggable detectors: Vulnerability Scanner (CVE), Misconfiguration Scanner (IaC), Secret Scanner, License Scanner, and the SBOM Generator (CycloneDX/SPDX).
Trivy Engine — Logic	Analysis stages applied to extracted artifacts: package analysis, layer analysis, resource analysis, and compliance evaluation against built-in or custom Rego policies.
Knowledge Base & Cache	The Trivy DB (Bolt DB packaged as an OCI artifact) aggregated from NVD, GitHub Security Advisories, OSV, and vendor feeds, plus the Policy Bundle. Locally cached under ~/.cache/trivy and refreshed every six hours by default.
Scan Cache	Per-layer analysis results keyed by layer digest. Re-scanning an image only re-analyses layers that changed. This is the single biggest CI speed-up.

Layer	Role in the scan flow
Outputs & Actions	Reports (table, JSON, SARIF, HTML, PDF), SBOM files (CycloneDX, SPDX), Kubernetes CRDs (VulnerabilityReport, ConfigAuditReport), pipeline pass/fail exit codes, and alerts.

4.1. End-to-End Scan Flow



Stage 1: Targets & Integrations (Input Ingestion)

The process begins when a specific target is selected for scanning. Trivy is highly versatile and handles distinct targets using the same underlying engine, driven by the specific flag passed to the CLI:

- **trivy image**: Target is handled as a stack of layers pulled from a remote registry, Docker daemon, containerd snapshot, or a local tarball.
- **trivy fs:** Target is treated as a walkable directory, traversing a local filesystem or git checkout to analyze lockfiles and raw source trees.
- **trivy config:** Target isolates Infrastructure as Code (IaC) configuration files, parsing patterns in Terraform (HCL), Kubernetes YAML, Dockerfiles, CloudFormation, and Helm charts.

- **trivy k8s / trivy aws:** Target connects externally to inventory live resources inside an active Kubernetes cluster or AWS account.

Stage 2: Integration Interfaces (Execution Context)

Trivy executes within three primary runtime contexts using the exact same binary:

- **CI/CD Pipeline:** Triggered automatically on every build runner (e.g., GitHub Actions, GitLab CI, Jenkins).
- **Developer Local Environment:** Run manually as a pre-commit check on a local terminal.
- **Kubernetes Operator**: Run continuously as a cluster controller via trivy-operator to actively scan live workloads.

Operational Mode Note: Trivy can run in Standalone Mode or Client-Server Mode. In client-server mode (activated via --server), thin Trivy clients connect over HTTPS to a central Trivy Server that acts as a shared cache and holds the vulnerability database. This is highly useful for air-gapped environments or preventing repetitive DB downloads across multi-runner CI systems.

Stage 3: Orchestration (CLI / Operator / Action)

Once invoked, control converges on the orchestrator. Regardless of the entry point, this layer acts as the centralized manager that handles the following execution workflow:

1. Parses user-defined arguments and configurations (such as severity thresholds, ignore files, output formats, and custom policies).
2. Constructs the definitive scan plan.
3. Evaluates database freshness: if the local database cache is older than the refresh threshold (six hours by default), the orchestrator automatically updates it.
4. Hands off the target to the artifact handler and drives the core scan engine.

Stage 4: Artifact Handling and the Scan Cache

Before any vulnerability or logic analysis can run, the artifact handler converts the raw target into a uniform, walkable filesystem:

- For container images, it pulls, unpacks, and decompresses each separate layer to present the resulting directory structure to the analyzers.
- For IaC configurations or code repos, it directly walks the directory tree.

To optimize execution speed, this layer heavily interfaces with the Scan Cache (located locally under ~/.cache/trivy). Per-layer analysis results are stored and keyed by their unique layer digest. If a base image has been scanned previously, only newly modified layers are re-analyzed, making this cache preservation the most critical optimization for CI performance.

Stage 5: Core Scanning Modules

Once the filesystem is exposed, specialized scanning modules extract raw security evidence. Each module runs strictly when relevant to the target:

- **Vulnerability Scanner (CVE):** Inventories installed components and pinned versions by reading OS package databases (dpkg, rpm, apk) and application-level language lockfiles (package-lock.json, requirements.txt, go.mod, pom.xml, Cargo.lock, etc.).

- **Misconfiguration Scanner (IaC):** Parses declarative configurations (Terraform, Kubernetes manifests, Dockerfiles, CloudFormation, Helm charts) into a structured format queries can read.

- **Secret Scanner:** Uses regex rules and entropy thresholds across file contents to detect hardcoded credentials, such as embedded AWS keys, GitHub tokens, and private keys.

- **SBOM Generator:** Generates a standard Software Bill of Materials (in CycloneDX or SPDX formats) to serve as a verifiable supply-chain artifact.

- **License Scanner:** Inspects third-party package licenses to flag items that breach organizational legal policies.

Stage 6: The Trivy Engine (Logic Block)

With the raw structural inventory built by the scanners, the Logic block processes the elements to map their exact context and origins:

- **Package Analysis:** Establishes exactly which packages exist and details where they came from.

- **Layer Analysis:** Pinpoints the exact container image layer that introduced each specific package or vulnerability.

- **Resource Analysis:** Evaluates parsed IaC resources alongside their specific security attributes.

- **Compliance Rules:** Executes Open Policy Agent (OPA) Rego policy evaluations directly against the parsed data.

The output of this block is a clean, structured set of findings where every risk is tagged with a precise severity level, source, and path location.

Stage 7: Detection (Knowledge Base & Cache)

- **Praveen Kankatala**

To determine if the structured inventory poses an actual risk, the findings are matched against two local validation stores:

- **Trivy DB:** A compact local BoltDB file containing aggregated advisories from NVD, GitHub Security Advisories, Google OSV, and vendor feeds (Red Hat, Debian, Alpine, Ubuntu, Amazon, SUSE, plus language ecosystems). Aqua publishes this daily as an open-source OCI artifact on the GitHub Container Registry (GHCR). Because lookups are executed against this local 30–60 MB cache file, scans can run entirely offline fast.

- **Policy Bundle:** A packaged collection of Rego rules covering CIS Benchmarks, AWS best practices, and Kubernetes hardening. Custom corporate compliance rules can be injected dynamically using the --config-policy flag.

Java Identification Exception: For application stacks with nested dependencies, Trivy utilizes a separate local store named trivy-java-db. This maps raw JAR file hashes back to official Maven coordinates to accurately identify unlabelled fat JARs.

Stage 8: Outputs & Actions (Reporting & Enforcement)

In the final stage, the orchestrator formats the structured findings and executes programmatic pipeline controls:

- **Format Reporting:** The Reporter renders the data for its target audience—colored terminal tables for human reading, JSON or SARIF for automated CI integration or code-scanning dashboards (e.g., GitHub Advanced Security), HTML or PDF for stakeholders, and SPDX/CycloneDX files for supply-chain analysis.

- **Kubernetes Mutation:** If executing under trivy-operator, results are translated into the cluster native API as VulnerabilityReport and ConfigAuditReport Custom Resources (CRDs) for monitoring dashboards to review.

- **Pipeline Policy Gates:** Finally, the orchestrator evaluates the findings against configurations like --exit-code and --severity. If a vulnerability or compliance gap meets or exceeds the defined threat gate (e.g., HIGH or CRITICAL), Trivy sets a non-zero process exit code (such as exit 1), successfully breaking the CI/CD pipeline and preventing insecure code from advancing to production.

5. Basic Image Scanning

Container image scanning is the most common use case, and it is the easiest place to start. Trivy can scan any image whether it lives on Docker Hub, in AWS ECR, in Google Artifact Registry, or in a private internal registry.

Scanning a Public Image

Give Trivy an image name. On the first run it will download the vulnerability database, which is then cached for next time.

```
trivy image python:3.10
```

Reading the Report

A typical scan groups findings into OS-level issues (things like glibc or openssl) and language-level issues (things in your pip or npm packages). For each finding you will see:

- **CVE ID:** the unique identifier.
- **Package:** the affected component.
- **Installed Version:** what is currently inside the image.
- **Fixed Version:** the version where the issue is patched. If this is empty, there is no fix yet.
- **Severity:** Low, Medium, High, or Critical.
- **Title and Link:** a short description and a URL with details.

Scanning a Private Image

Trivy works the same way for private images. You just need to be logged in to the registry first.

```
docker login registry.mycompany.com
trivy image registry.mycompany.com/team/app:1.0.0
```

> **Pro tip:** Scan two versions of the same image — an older tag and a newer one — and compare. The difference in findings tells you exactly how much risk you can remove just by upgrading your base image. Sometimes that one number changes the conversation with your team faster than any security policy ever could.

6. Essential Flags You Will Use Every Day

Trivy's defaults are deliberately verbose because the tool does not know what you care about. A handful of flags will turn it from a firehose into something useful for everyday work.

6.1 Filter by Severity (--severity)

By default, Trivy prints everything from Low to Critical. The --severity flag accepts a comma-separated list and shows only those levels.

```
trivy image --severity HIGH,CRITICAL python:3.10
```

■ **Praveen Kankatala**

6.2 Choose What to Scan (--vuln-type)

If you only care about OS-level problems or only about language libraries, narrow the scope.

```
trivy image --vuln-type os python:3.10
trivy image --vuln-type library python:3.10
```

6.3 Fail the Build (--exit-code)

By default, Trivy exits with code 0 even when it finds problems. Fine when you read the report yourself; dangerous in a pipeline because nothing stops the build. Setting --exit-code to 1 makes the step fail when matching findings exist.

```
trivy image --severity HIGH,CRITICAL --exit-code 1 python:3.10
```

6.4 Hide Unfixable Issues (--ignore-unfixed)

This is one of the highest-value flags in the entire tool, and most people discover it too late. By default, Trivy reports every vulnerability, including ones where no patch exists yet. Those findings are not actionable there is literally nothing you can do about them until upstream releases a fix.

Adding --ignore-unfixed cuts out that noise and leaves you with the findings you can actually do something about today.

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed python:3.10
```

> **When to use it:** Almost always in CI/CD. It keeps your pipeline focused on "there is a fix, please apply it" instead of "here is a list of things the world has not solved yet." In a vulnerability tracking dashboard, you may still want the full picture.

6.5 Skip Paths (--skip-dirs and --skip-files)

Sometimes your repo contains directories you really do not want scanned test fixtures, vendored examples, sample data with intentionally fake credentials. These flags let you exclude them.

```
trivy fs \
 --skip-dirs ./tests/fixtures \
 --skip-dirs ./vendor \
 --skip-files README.md \
 .
```

7. End-to-End: Securing a Local Docker Image

This is the shift-left workflow find and fix issues on your own machine before the image ever reaches a registry or a production cluster. The earlier you find a problem, the less it costs to fix.

Step 1: Create the Application

```
# app.py
print("Hello there")
```

Step 2: Write the Dockerfile

Older base images often carry known vulnerabilities, which is exactly why we are scanning. Expect to see findings on the first run.

```
FROM python:3.10-slim
RUN pip install flask
COPY app.py /app.py
CMD ["python", "app.py"]
```

Step 3: Build and Scan

```
docker build -t app-with-issue .
time trivy image app-with-issue:latest
```

The time command tells you how long the scan took, which is handy when comparing local scans against server-mode scans later.

Step 4: Remediate

Look at the report, find the package that needs upgrading, and bake the upgrade into the Dockerfile so it happens at build time.

```
FROM python:3.10-slim
RUN pip install --upgrade pip
RUN pip install flask
COPY app.py /app.py
CMD ["python", "app.py"]
```

Step 5: Rebuild and Verify

```
docker build -t app-with-fix .
trivy image app-with-fix
```

If the original finding is gone, the fix worked. If it is still there, the package probably needs an explicit upgrade or a newer base image entirely.

Working With Alpine Images

Alpine uses apk instead of apt or yum. Once you know which library is vulnerable (libpng, busybox, openssl), upgrade it inside the Dockerfile.

```
RUN apk update && apk upgrade --no-cache libpng busybox
```


- **Praveen Kankatala**

Building on an ARM machine (like Apple Silicon) and pushing to an x86 registry? Set the platform explicitly to avoid mismatch errors.

```
docker build --platform linux/arm64 -t myimage:v2 .
```

8. Generating HTML Reports

Terminal output is fine for developers, but managers, auditors, and security teams usually want something more polished. Trivy can produce an HTML report using an official template that looks professional out of the box.

Step 1: Download the Template

```
curl -o html.tpl \
 https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/html.tpl
```

Step 2: Run the Scan With the Template

```
trivy image \
 --format template \
 --template @html.tpl \
 -o trivy.html \
 python:3.10-slim
```

Step 3: Open the Report

Open trivy.html in any browser. You get a sortable table, color-coded severity, the scan timestamp, and links to detailed CVE pages. It is the kind of report you can drop into a compliance package without apologizing for the formatting.

```
Start trivy.html
```

9. Secrets Scanning

Secrets are the sensitive values that keep your systems running API keys, access tokens, passwords, private keys. When they accidentally end up in source control, the worst case is genuinely bad: attackers stealing data, running up cloud bills in the millions, or taking over accounts entirely. Trivy can catch many common types of secrets before they reach a shared repository.

Why Secrets Leak

- Developers hardcode tokens during testing and forget to remove them.
- Config files with real credentials get committed by accident.
- Templates and samples are copied as-is and never sanitized.

How Trivy Finds Them

Trivy uses pattern matching, not AI. It applies regular expressions and entropy checks to look for shapes that match known credential formats AWS access keys start with AKIA, GitHub tokens start with ghp_, and so on. It scans .env files, JSON, YAML, source code, Dockerfiles, and documentation.

What Trivy Does Not Do

- It does not contact AWS, GitHub, or any other provider to check whether a key is still active.
- It does not know if a flagged string is a real key or just an example.
- It does not rotate, revoke, or delete leaked credentials. That is still your job.

Running a Secrets Scan

```
trivy fs --scanners secret .
```

To reduce noise from documentation that intentionally contains fake examples, skip specific files.

```
trivy fs --scanners secret --skip-files README.md .
```

> **If you find a real one:** Removing the file from the repo is not enough — git history still has it. Rotate the credential immediately, then clean the history (with tools like git-filter-repo or BFG), and assume the old secret is already compromised.

10. Repository and Filesystem Scanning

Trivy can scan source repositories as well as plain folders on disk. This is useful for catching vulnerabilities, misconfigurations, and secrets early, before the code is even built into an image.

Scanning a Local Repository

```
trivy repo .
trivy repo --severity CRITICAL .
```

Scanning a Remote GitHub Repository

No need to clone first. Trivy can read a public GitHub URL directly.

```
trivy repo https://github.com/example/my-app
trivy repo --severity HIGH https://github.com/example/my-app
```

What Gets Audited

- Infrastructure-as-Code files for misconfigurations (Terraform, Kubernetes, CloudFormation, Helm, Dockerfiles).

- Application dependency files (requirements.txt, package.json, go.mod, pom.xml, Gemfile.lock) for vulnerable libraries.
- All scanned files for hardcoded secrets.

11. Software Bill of Materials (SBOM)

A Software Bill of Materials is the ingredient list for your software. It records every component, its version, and where it came from. SBOMs are increasingly required by regulators and large enterprise customers because they make it possible to answer the question "are we affected by this new vulnerability?" in minutes instead of weeks.

CycloneDX Format (Security-Focused)

CycloneDX is widely used in DevSecOps for vulnerability management.

```
trivy fs --format cyclonedx --output sbom.json .
```

SPDX Format (Compliance-Focused)

SPDX is maintained by the Linux Foundation and is generally preferred for license auditing and legal review.

```
trivy fs --format spdx --output sbom.spdx .
```

Generating an SBOM From an Image

```
trivy image --format cyclonedx --output image-sbom.json myimage:1.0
```

Scanning an Existing SBOM

Here is something a lot of people miss: once you have an SBOM, Trivy can scan that file directly without re-analyzing the original image or codebase. This is incredibly useful for a few scenarios:

- Auditing third-party software when you do not have the source code, just the SBOM the vendor provided.
- Re-checking an old build for new vulnerabilities, because new CVEs are discovered every day.
- Speeding up compliance reviews you scan the SBOM artifact instead of pulling the image again.

```
trivy sbom sbom.json
```

The output looks just like a regular vulnerability scan, but the input was the bill of materials rather than the software itself.

Best Practices

- Regenerate the SBOM with every build. An outdated SBOM gives a false sense of security.
- Store SBOMs alongside the build artifact so they can be retrieved later.
- Use CycloneDX for vulnerability work and SPDX for license and legal work.

12. Kubernetes Cluster Scanning

Container security does not end when the image is built. A perfectly secure image can still be deployed into a cluster with sloppy RBAC, missing network policies, or pods running as root. The trivy k8s command scans live clusters to find both kinds of problems vulnerable workloads and misconfigured resources in one shot.

What It Looks At

- Images currently running in the cluster, scanned for vulnerabilities.
- Pods, deployments, and other workloads for misconfigurations like privileged containers or missing resource limits.
- Cluster-level resources like RBAC roles, network policies, and service accounts.
- Secrets stored as Kubernetes Secret objects.

Scanning the Whole Cluster

Trivy uses your current kubeconfig context, so whatever cluster kubectl talks to is what trivy k8s will scan.

```
trivy k8s --report summary
```

A summary report gives you a high-level view across namespaces, severities, and resource types. For a deeper dive on specific resources, ask for the detailed report.

```
trivy k8s --report all
```

Scoping the Scan

Whole-cluster scans can be slow and noisy on large clusters. Narrow the scope to a namespace or resource type.

```
trivy k8s --namespace production --report summary
trivy k8s deployment/my-app --report all
trivy k8s --namespace production --severity CRITICAL
```

Continuous Scanning With the Trivy Operator

Running trivy k8s by hand is fine for spot checks, but real environments need continuous coverage. The Trivy Operator is a Kubernetes controller that does exactly that: install it once, and it automatically scans every workload as it gets deployed or updated, storing results as custom resources you can query with kubectl.

```
# Install with Helm
helm repo add aqua https://aquasecurity.github.io/helm-charts/
helm install trivy-operator aqua/trivy-operator \
 --namespace trivy-system \
 --create-namespace

# Query results like any Kubernetes resource
kubectl get vulnerabilityreports --all-namespaces
kubectl get configauditreports --all-namespaces
```

> **Why this matters:** Manual scans answer "is my cluster secure right now?" The operator answers "is my cluster staying secure as people deploy things to it?" That is a much more useful question to have an automatic answer to.

13. Cloud Account Scanning (AWS)

Trivy can also scan your AWS account directly. This is a different kind of scan instead of looking inside an image or a file, it queries AWS APIs to inspect your real resources and check them against security best practices.

What It Checks

- S3 buckets — public access, encryption at rest, logging.
- IAM — overly permissive policies, root account usage, missing MFA.
- EC2 — security group rules, unencrypted volumes, public IPs.
- RDS, EKS, Lambda, CloudFront, and many other services.

Running an AWS Scan

Trivy uses your existing AWS credentials (the same ones the AWS CLI uses), so make sure you are configured first.

```
# Scan the entire account
trivy aws --region us-east-1

# Scan only one service

trivy aws --region us-east-1 --service s3
```

- **Praveen Kankatala**

```
# Scan against a specific compliance benchmark
trivy aws --region us-east-1 --compliance aws-cis-1.5
```

> **Heads up:** Use a read-only IAM role for scanning. Trivy does not need write access to do its job, and giving any scanner more permissions than it needs is exactly the kind of thing it would flag if you pointed it at itself.

14. Virtual Machine Image Scanning

Containers are not the only thing that ships as images. Cloud VMs, on-prem hypervisors, and golden images for desktops all package up an operating system and software. The trivy vm command scans those images using the same vulnerability database it uses for containers.

Supported Formats

- AMI — Amazon Machine Images.
- VMDK — VMware disk images.
- OVA — Open Virtualization Appliance bundles.
- Raw disk images and various other VM formats.

Scanning a Local VM Image

```
trivy vm disk.vmdk
trivy vm --severity HIGH,CRITICAL ubuntu.ova
```

Scanning an AWS AMI

```
trivy vm ami:ami-0123456789abcdef0
```

This is genuinely useful if you maintain golden AMIs for your fleet. You can scan a base AMI before promoting it and catch problems before they propagate to hundreds of running instances.

15. Configuration and Control

Once you have been running Trivy for a while, you will start to want more control over which findings appear, how the command is configured, and how policies are enforced. This chapter covers the tools that take Trivy from "scanner that prints stuff" to "scanner that fits your team's workflow."

.trivyignore — Suppress Reviewed CVEs

Not every finding is something you can or should fix immediately. Sometimes a CVE does not actually apply to your usage. Sometimes the fix is in flight but not yet released. Sometimes a third-party

dependency has a CVE you have accepted as a known risk. The .trivyignore file lets you suppress specific CVEs without losing track of them.

Create a file named .trivyignore in your project root with one CVE ID per line. You can add comments to explain why each one is ignored — and please do, because the next person looking at this file will thank you.

```
# .trivyignore

# Vulnerable code path is not reachable in our config
CVE-2023-12345

# Fix is in upstream main, awaiting next release - review by 2026-06-01
CVE-2023-67890

# Accepted risk - documented in SECURITY.md
CVE-2024-11111
```

Trivy will quietly drop those CVEs from future scans. You still see everything else.

> **Discipline matters here:** Treat .trivyignore like code. Review it in pull requests, add expiry dates in the comments, and revisit it during release cycles. An ignore file that nobody ever reviews becomes a graveyard of security debt.

trivy.yaml — Stop Repeating Yourself

If you find yourself typing the same flags over and over same severity filter, same skip dirs, same output format pull them into a config file. Trivy automatically looks for trivy.yaml in the current directory.

```
# trivy.yaml
severity:
  - HIGH
  - CRITICAL

vulnerability:
  ignore-unfixed: true

scan:
  skip-dirs:
    - ./tests/fixtures
    - ./vendor

format: table
exit-code: 1

cache:
  dir: /var/cache/trivy
```

- **Praveen Kankatala**

With this file in place, your command shrinks to just the target.

```
trivy image myapp:latest
```

All those settings apply automatically. Commit the file to your repo so everyone on the team scans the same way.

Policy as Code With Rego and OPA

For misconfiguration scanning, Trivy comes with a library of built-in rules things like "S3 buckets should not be public" and "pods should not run as root." But every organization has its own rules. Maybe you require a specific tag on every AWS resource, or you forbid certain image registries, or your Kubernetes pods must always have liveness probes.

Trivy supports custom policies written in Rego, the same language used by Open Policy Agent (OPA). Write the policy, point Trivy at it, and your rules get checked alongside the built-in ones.

```
# Example Rego policy: forbid latest tag
package custom.dockerfile.latest_tag

deny[msg] {
  input.Stages[_].Commands[_].Cmd == "from"
  endswith(input.Stages[_].Commands[_].Value[0], ":latest")
  msg := "Do not use the 'latest' tag in production Dockerfiles"
}

trivy config \
  --policy ./custom-policies/ \
  --namespaces custom \
  ./Dockerfile
```

This is how you turn organizational standards into automated checks instead of wiki pages no one reads.

16. Database Management

Trivy's speed comes from its local database. Most of the time you do not need to think about it. But there are situations air-gapped environments, slow networks, or Java-heavy projects — where understanding how the database works pays off.

Pre-downloading the Database

In an air-gapped environment, your scan host cannot reach the internet to grab the database on the fly. You need to download it somewhere with network access and copy it across.

```
# On a machine with internet access
trivy image --download-db-only
```

```
# The database lands in ~/.cache/trivy/db/ by default
# Tar it up and ship it to the air-gapped host
tar czf trivy-db.tar.gz ~/.cache/trivy/db/
```

Running Without Network Access (--offline-scan)

Once the database is in place on your air-gapped host, tell Trivy not to attempt any network calls during the scan.

```
trivy image --offline-scan myimage:1.0
```

Without this flag, Trivy may try to contact the internet for things like Java dependency resolution or database freshness checks, and the scan will fail in a confusing way when the network call times out.

The Java DB

Java is a special case. The Maven ecosystem is huge and the metadata needed to map JAR files back to their official coordinates is enormous, so Trivy keeps it in a separate database that is downloaded on demand the first time you scan a Java project.

In normal environments this just happens. In air-gapped environments, you need to download it explicitly the same way you did the main database.

```
trivy image --download-java-db-only
```

> **Common gotcha:** Teams set up air-gapped scanning, test it on a Python image, and everything works. Then someone scans a Java image and the scan hangs trying to download the Java DB. Always pre-download both databases when going air-gapped.

Air-Gapped Installation Summary

The full air-gapped setup, in order:

1. Install the Trivy binary on the air-gapped host (download it from a machine with internet and copy it over).
2. Download both databases on a connected machine: trivy image --download-db-only and trivy image --download-java-db-only.
3. Copy the cache directories to the same path on the air-gapped host.
4. Always scan with --offline-scan so Trivy never attempts network calls.
5. Refresh both databases on a regular schedule — once a day is typical, once a week is the absolute minimum.

17. Automating Scans With a Shell Script

- **Praveen Kankatala**

Once you find yourself running the same Trivy command repeatedly, wrap it. The script below asks for an image name, creates a timestamped report, and stores everything in a dedicated folder.

scan.sh

```
#!/bin/bash

read -p "Enter image name : " IMAGE
DATE=$(date +"%Y-%m-%d_%H-%M")
REPORT_DIR="trivy-reports"

mkdir -p "$REPORT_DIR"

echo "Starting Trivy scan for $IMAGE"
echo "Scan time: $DATE"

trivy image \
 --severity HIGH,CRITICAL \
 --ignore-unfixed \
 --format table \
 "$IMAGE" > "$REPORT_DIR/trivy-scan-$DATE.txt"

echo "Scan completed"
echo "Report saved to $REPORT_DIR/trivy-scan-$DATE.txt"
```

```
chmod +x scan.sh
./scan.sh
```

18. Pre-commit Hook for Secrets

The cheapest place to catch a leaked secret is on the developer's laptop, before the commit ever happens. A pre-commit hook runs Trivy automatically when someone tries to commit and blocks the commit if anything looks like a secret.

Using the pre-commit Framework

The pre-commit framework (pre-commit.com) is the easiest way to set this up consistently across a team. Create a .pre-commit-config.yaml file in your repo:

```
repos:
 - repo: https://github.com/aquasecurity/trivy
   rev: v0.50.0  # use the latest stable tag
   hooks:
     - id: trivy-fs
       args:
         - --scanners
```

```
      - secret
      - --exit-code
      - "1"
      - --skip-dirs
      - tests/fixtures
```

Install the hook into each developer's local Git workflow:

```
pip install pre-commit
pre-commit install
```

From now on, every git commit will run Trivy's secret scanner first. If it finds something, the commit is blocked until the developer either removes the secret or (after careful review) adds it to .trivyignore.

A Simpler Plain Git Hook

If you do not want to add the pre-commit framework, here is the same idea as a hand-written hook. Save it as .git/hooks/pre-commit and make it executable.

```
#!/bin/bash
echo "Running Trivy secret scan..."
trivy fs --scanners secret --exit-code 1 --quiet .
if [ $? -ne 0 ]; then
 echo "Secrets detected. Commit blocked."
 exit 1
fi
```

19. CI/CD Integration With Jenkins

In a real pipeline you usually want to build the image, scan it, save the report as an artifact, and (optionally) fail the build for High or Critical findings. The Jenkinsfile below shows the full pattern, including JSON, HTML, and SBOM outputs all from a single workspace.

```
pipeline {
   agent any

   environment {
      APP_DIR = "9-trivy-artifact-project"
      DOCKER_IMAGE = "python-trivy-demo"
      IMAGE_NAME = "${DOCKER_IMAGE}:${BUILD_NUMBER}"
      TRIVY_CACHE_DIR = "${WORKSPACE}/.trivy_cache"
   }

   stages {
      stage('Checkout') {
         steps { checkout scm }
      }
```

```
    stage('Build Docker Image') {
        steps {
            dir("${APP_DIR}") {
                sh "docker build -t ${IMAGE_NAME} ."
            }
        }
    }

    stage('Trivy Image Scan') {
        steps {
            sh "mkdir -p ${TRIVY_CACHE_DIR} reports"

            dir("${APP_DIR}") {
                sh """
                trivy image \
                  --cache-dir ${TRIVY_CACHE_DIR} \
                  --format json \
                  --output ${WORKSPACE}/reports/trivy-image.json \
                  ${IMAGE_NAME}
                """

                sh """
                trivy image \
                  --cache-dir ${TRIVY_CACHE_DIR} \
                  --format template \
                  --template @trivy/html.tpl \
                  --output ${WORKSPACE}/reports/trivy-image.html \
                  ${IMAGE_NAME}
                """

                sh """
                trivy image \
                  --cache-dir ${TRIVY_CACHE_DIR} \
                  --format cyclonedx \
                  --output ${WORKSPACE}/reports/sbom.json \
                  ${IMAGE_NAME}
                """
            }
        }
    }
}

post {
    always {
        archiveArtifacts artifacts: 'reports/*', fingerprint: true
    }
}
```

- **Praveen Kankatala**

```
}
```

Pipeline Tips

- Set a shared --cache-dir so the vulnerability database is downloaded once per agent and reused across builds.
- Always archive the report files so failed builds can still be investigated later.
- Add --exit-code 1 --severity HIGH,CRITICAL to a separate stage if you want to block deployment on serious findings while still publishing the full report.

20. CI/CD Integration With GitHub Actions

GitHub Actions has an official Trivy action. This is the easiest way to add scanning to any GitHub repository. The example below scans the built image and uploads results in SARIF format so they appear directly in the repository's Security tab.

```
name: Trivy Image Scan

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build image
        run: docker build -t myapp:${{ github.sha }} .

      - name: Run Trivy vulnerability scanner
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'myapp:${{ github.sha }}'
          format: 'sarif'
          output: 'trivy-results.sarif'
          severity: 'HIGH,CRITICAL'
          ignore-unfixed: true
          exit-code: '1'

      - name: Upload Trivy results to GitHub Security tab
        if: always()
        uses: github/codeql-action/upload-sarif@v3
```

```
      with:
        sarif_file: 'trivy-results.sarif'
```

21. CI/CD Integration With GitLab CI

GitLab CI integrates beautifully with Trivy through the official container image. Trivy can output results in GitLab's native container scanning format, which means findings show up in merge requests with the same UI as GitLab's built-in scanners.

```
# .gitlab-ci.yml
stages:
  - build
  - scan

variables:
  IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHA
  TRIVY_CACHE_DIR: ".trivycache/"

build:
  stage: build
  image: docker:24
  services:
    - docker:24-dind
  script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
    - docker build -t $IMAGE_TAG .
    - docker push $IMAGE_TAG

trivy_scan:
  stage: scan
  image:
    name: aquasec/trivy:latest
    entrypoint: [""]
  cache:
    paths:
      - .trivycache/
  script:
    - trivy image
      --severity HIGH,CRITICAL
      --ignore-unfixed
      --exit-code 1
      --format template
      --template "@contrib/gitlab.tpl"
      --output gl-container-scanning-report.json
      $IMAGE_TAG
  artifacts:
    when: always
```

```
  reports:
    container_scanning: gl-container-scanning-report.json
```

With the container_scanning report type, GitLab automatically displays findings as a security widget in the merge request, with a diff between what is on the branch and what is on main.

22. CI/CD Integration With Azure DevOps

Azure DevOps does not have a first-party Trivy task, but the official Trivy container image works perfectly inside a standard pipeline. Here is a working azure-pipelines.yml that builds an image and scans it.

```
# azure-pipelines.yml
trigger:
  - main

pool:
  vmImage: 'ubuntu-latest'

variables:
  imageName: 'myapp:$(Build.BuildId)'

steps:
  - task: Docker@2
    displayName: 'Build image'
    inputs:
      command: 'build'
      Dockerfile: 'Dockerfile'
      tags: '$(Build.BuildId)'
      repository: 'myapp'

  - script: |
      docker run --rm \
        -v /var/run/docker.sock:/var/run/docker.sock \
        -v $(Pipeline.Workspace)/.trivycache:/root/.cache/ \
        aquasec/trivy:latest image \
          --severity HIGH,CRITICAL \
          --ignore-unfixed \
          --exit-code 1 \
          --format json \
          --output trivy-report.json \
          $(imageName)
    displayName: 'Trivy security scan'

  - task: PublishBuildArtifacts@1
    condition: always()
    displayName: 'Publish scan report'
```

```
inputs:
  pathToPublish: 'trivy-report.json'
  artifactName: 'TrivySecurityReport'
```

23. IDE Integration: VS Code Extension

If you want feedback even earlier than commit time, install the Trivy VS Code extension. It scans your workspace in the background and shows vulnerabilities, misconfigurations, and secrets directly in the editor right next to the code that caused them.

Installing the Extension

- Open the Extensions view in VS Code (Ctrl+Shift+X).
- Search for "Trivy" — the publisher is Aqua Security.
- Click Install.
- Make sure the trivy binary is installed on your PATH; the extension uses it under the hood.

What You Get

- A Trivy panel listing every finding in the workspace, grouped by file.
- Inline squiggles in the editor on lines with issues, just like syntax errors.
- One-click navigation from a finding to the exact line of code.
- Configurable severity filter so you can hide Low and Medium when you just want to focus on the bad ones.

> **Why bother:** The faster a developer sees a finding, the more likely they are to fix it. Catching a misconfiguration as they type it is much better than catching it in a pull request, and dramatically better than catching it in production.

24. Compliance Scanning

Compliance is often where security meets paperwork. Auditors do not just want to know that you are "secure" they want to see that you check against specific, recognized frameworks. Trivy has built-in support for the major ones.

Supported Frameworks

- CIS Benchmarks — Docker, Kubernetes, AWS, and others. These are the de facto industry standard for hardening checklists.
- NSA / CISA Kubernetes Hardening Guidance.

- PCI-DSS for any system that handles payment card data.
- HIPAA for healthcare workloads.
- Custom benchmarks you can define in YAML if your organization has its own controls.

Running a Compliance Scan

```
# Docker CIS benchmark on an image
trivy image --compliance docker-cis-1.6.0 myimage:latest

# Kubernetes CIS benchmark on a live cluster
trivy k8s --compliance k8s-cis-1.23 --report summary

# NSA hardening guidance for Kubernetes
trivy k8s --compliance k8s-nsa-1.0 --report summary

# AWS CIS benchmark on a cloud account
trivy aws --compliance aws-cis-1.5 --region us-east-1
```

The output is structured around the benchmark itself — each control with a pass or fail and links to the official documentation — which makes it much easier to hand to an auditor than a raw vulnerability list.

25. Reporting Formats

Different audiences want different reports. Developers like terminal tables, security teams like SARIF, dashboards like JSON, and CI systems like JUnit. Trivy speaks all of them.

SARIF — for GitHub, Azure DevOps, and Security Tools

SARIF (Static Analysis Results Interchange Format) is the standard JSON format for static analysis findings. GitHub's Security tab, Azure DevOps, and many SIEM tools ingest SARIF natively.

```
trivy image --format sarif --output trivy.sarif myimage:1.0
```

JUnit — for CI Test Dashboards

Most CI systems already know how to display JUnit XML. Outputting findings in this format means they appear in the same test results UI everyone already uses, alongside unit tests.

```
trivy image --format junit --output trivy-junit.xml myimage:1.0
```

Suddenly your security findings live in the same dashboard as your failing tests, which makes them much harder to ignore.

JSON — for Custom Tooling

If you are building your own dashboards or feeding results into another system, JSON is the universal format.

```
trivy image --format json --output trivy.json myimage:1.0
```

--list-all-pkgs — Include Clean Packages

By default Trivy only reports packages that have findings. Sometimes — for SBOM generation, software inventory, or detailed audit evidence — you want every package listed, even the ones with no known vulnerabilities.

```
trivy image --format json --list-all-pkgs --output full-inventory.json myimage:1.0
```

26. License Scanning

Open source licenses sound boring until your legal team asks "which of our products contain GPL code?" the day before a release. Trivy can scan your dependencies and report on the licenses they use, which makes those conversations dramatically easier.

Running a License Scan

```
# License scanning on a filesystem
trivy fs --scanners license .

# License scanning on a container image
trivy image --scanners license myimage:1.0
```

Why This Matters

- Permissive licenses like MIT and Apache are usually fine to redistribute.
- Copyleft licenses like GPL and AGPL come with obligations that may require you to publish source code.
- Unknown or custom licenses are a red flag that needs human review.

Trivy assigns each license a confidence level and a category, which gives your legal team a quick triage view instead of a wall of raw license strings.

27. Misconfiguration Scanning

Vulnerabilities are about broken code. Misconfigurations are about insecure settings. Trivy treats them as separate scanners because they look at different things and need different rules.

What It Checks

- Dockerfile — using latest tag, running as root, exposing unnecessary ports.
- Kubernetes manifests — missing resource limits, privileged containers, hostPath mounts, missing security contexts.
- Terraform — S3 buckets without encryption, security groups open to the world, missing logging.
- CloudFormation — same kinds of issues as Terraform, just in a different format.
- Helm charts — both the chart itself and the rendered manifests.

Running a Misconfig Scan

```
# Scan a directory for any misconfigurations
trivy config .

# Or use the unified fs scanner
trivy fs --scanners misconfig .

# Scan only Dockerfiles
trivy config --file-patterns "dockerfile:Dockerfile*" .
```

Each finding includes the file, the line, the rule that was violated, the severity, and a suggestion for how to fix it. This is the kind of scanning that catches problems while you are still writing the config, which is exactly when they are easiest to fix.

28. Trivy Server Mode for Enterprises

When dozens of pipelines all run Trivy independently, each one downloads its own copy of the vulnerability database. That wastes bandwidth and disk, and worse, can produce inconsistent results when different agents have different database versions.

Server mode solves this by running one central Trivy server that owns the database. All the clients — developer machines and CI/CD jobs — ask the server for results instead of running scans on their own.

Benefits

- Everyone gets identical results because everyone uses the same database.
- The database is updated in one place, cutting network traffic dramatically.
- CI/CD jobs start scanning immediately because they no longer wait for a database download.
- Auditors get a single point of reference for what was scanned and when.

Starting the Server

```
trivy server --listen 0.0.0.0:4954
```

Running a Client Scan

```
trivy image --server http://trivy-server.internal:4954 myimage:1.0
```

Performance Tuning

Trivy is fast out of the box, but a few knobs help at scale.

- **Parallel scans:** the --parallel flag controls how many files Trivy analyzes at once. The default works for most cases, but on big projects you can raise it to use more cores.
- **Resource limits:** in containerized environments, give Trivy enough CPU and memory. A starved scanner gets slow and confusing, not faster.
- **Cache sharing:** always use a shared --cache-dir on CI agents so the database is downloaded once per machine, not once per build.
- **Scope your scans:** if you only care about a namespace, scan that namespace. If you only care about HIGH and CRITICAL, filter at scan time. The cheapest finding to process is the one you never asked for.

When Server Mode Is Worth It

Server mode is recommended for organizations with many pipelines, regulated environments, or large teams. Solo developers and small teams who are still learning the tool will probably find plain local scans simpler and perfectly sufficient.

29. Trivy Compared to Other Scanners

Trivy is not the only container scanner. Two common alternatives are Grype and Docker Scout. Here is the short version of when each is a good fit.

Tool	Best For	Trade-off
Trivy	All-in-one scanning: images, filesystems, repos, IaC, secrets, SBOMs, Kubernetes, cloud	Best general-purpose starting point; minimal setup
Grype	SBOM-driven vulnerability analysis with high accuracy	Output is more technical; steeper learning curve
Docker Scout	Teams already living inside Docker Desktop	Requires a Docker account; slower for repeated CLI use

For most teams, Trivy is the right place to start. No account, straightforward command line, covers most needs in a single tool. Reach for Grype if your work is heavily SBOM-driven, or for Docker Scout if your team already lives inside Docker Desktop.

- **Praveen Kankatala**

30. Best Practices

These habits give you most of the value of Trivy with very little effort.

Daily Habits

- Scan early and scan often. Run Trivy locally before pushing, not only in CI.
- Focus first on High and Critical issues that already have a fix available — use --ignore-unfixed.
- Re-scan after every dependency change or base image bump.

Image Hygiene

- Pick small, well-maintained base images — official slim or alpine variants are a good start.
- Pin specific image digests in production for reproducible builds and predictable scan results.
- Avoid installing build tools in the final image. Use multi-stage builds to keep the runtime layer minimal.

Pipeline Hygiene

- Use --exit-code 1 with --severity HIGH,CRITICAL so insecure code cannot be deployed by accident.
- Cache the database directory between builds with --cache-dir to keep pipelines fast.
- Archive scan reports as build artifacts so they are available for later review and audits.

Secrets Hygiene

- Add Trivy secrets scanning to a pre-commit hook so credentials never reach the remote repo.
- If a real secret is found, rotate it immediately. Removing the file is not enough — Git history still has it.

Process Hygiene

- Treat .trivyignore like code. Review it in pull requests, add expiry dates, and revisit it during release cycles.
- Store shared settings in trivy.yaml and commit it. Everyone on the team should scan the same way.
- Generate SBOMs as part of every release build and store them alongside the artifact.

31. Common Trivy Commands Cheat Sheet

A quick reference of the commands you will reach for most often.

Scan a container image

```
trivy image python:3.10
```

Scan only fixable High and Critical issues, fail on findings

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 python:3.10
```

Scan a local folder

```
trivy fs .
```

Scan for secrets only

```
trivy fs --scanners secret .
```

Scan a remote GitHub repository

```
trivy repo https://github.com/example/my-app
```

Scan a Kubernetes cluster

```
trivy k8s --report summary
```

Scan an AWS account

```
trivy aws --region us-east-1
```

Scan a VM image

```
trivy vm disk.vmdk
```

Scan an existing SBOM file

```
trivy sbom sbom.json
```

Run a compliance check

```
trivy image --compliance docker-cis-1.6.0 myimage:1.0
```

Produce an HTML report

```
trivy image --format template --template @html.tpl -o report.html myimage
```

Produce a SARIF report (for GitHub Security tab)

```
trivy image --format sarif -o trivy.sarif myimage
```

Produce a JUnit report (for CI dashboards)

```
trivy image --format junit -o trivy-junit.xml myimage
```

Produce an SBOM in CycloneDX format

```
trivy fs --format cyclonedx --output sbom.json .
```

- **Praveen Kankatala**

Pre-download the database for air-gapped use

```
trivy image --download-db-only
trivy image --download-java-db-only
```

Run offline (air-gapped)

```
trivy image --offline-scan myimage
```

Use a central Trivy server

```
trivy image --server http://trivy-server:4954 myimage
```

32. Glossary

- **CVE:** Common Vulnerabilities and Exposures. A unique identifier for a publicly known security problem.
- **CVSS:** Common Vulnerability Scoring System. A numerical score from 0 to 10 indicating how severe a vulnerability is.
- **SBOM:** Software Bill of Materials. A machine-readable list of every component inside a piece of software.
- **IaC:** Infrastructure as Code. Files such as Terraform, Kubernetes manifests, or Dockerfiles that define infrastructure declaratively.
- **SARIF:** Static Analysis Results Interchange Format. A JSON standard used by GitHub and other tools to display scan results.
- **JUnit:** An XML format used by CI systems to display test results. Trivy can output findings in this format so they appear in test dashboards.
- **Rego:** A declarative policy language used by Open Policy Agent (OPA) and supported by Trivy for custom misconfiguration rules.
- **OPA:** Open Policy Agent. A general-purpose policy engine that uses Rego as its language.
- **Air-gapped:** A system that has no direct connection to the public internet. Common in regulated industries and high-security environments.
- **Shift Left:** Moving security checks earlier in the development lifecycle so problems are caught before deployment.
- **False Positive:** A finding the tool flagged as a problem but that is actually safe in your context.
- **Trivy Operator:** A Kubernetes controller that continuously runs Trivy scans on cluster workloads and stores the results as Kubernetes resources.

33. Trivy vs SonarQube

Trivy and SonarQube are often mentioned together because they both appear in the CI pipeline and both report "security" findings. In practice they solve very different problems and are complementary, not competing tools.

Dimension	Trivy	SonarQube
Primary purpose	Cloud-native security scanner: vulnerabilities, misconfigurations, secrets, SBOM, licenses.	Code quality and SAST platform: bugs, code smells, maintainability, security hotspots.
Category	SCA (Software Composition Analysis), container security, IaC scanning, secret detection.	SAST (Static Application Security Testing), code quality, technical debt management.
What it analyses	Compiled/packaged artefacts: container images, lockfiles, IaC files, K8s manifests, cloud APIs.	Source code itself: Java, C#, JavaScript, Python, Go, and 30+ other languages.
Detection method	Inventory + database lookup (package versions matched against CVE feeds) and Rego policy checks.	AST parsing, data-flow analysis, taint tracking, and pattern rules against source code.
Knowledge sources	NVD, GitHub Security Advisories, OSV, Red Hat, Debian, Alpine, Ubuntu, language ecosystems.	Built-in rule sets (Sonar Way), plus optional plugins like SonarLint, security rules, and OWASP mappings.
Architecture	Single Go binary runs anywhere. Optional client–server mode for shared DB cache.	Server + database (PostgreSQL) + scanners that push results to the server. Web UI for review.
Deployment	Local CLI, CI step, Kubernetes operator, GitHub Action — same binary, all modes.	Self-hosted server (SonarQube) or SaaS (SonarCloud). Scanners installed in CI/IDE.
Where in SDLC	Build → Image registry → Deploy → Runtime. Late stages — after code becomes artefacts.	Commit → Pull request → Build. Early stages — while code is being written.
Output	CLI table, JSON, SARIF, SBOM (CycloneDX/SPDX), K8s CRDs, exit codes.	Rich web dashboard with trends, quality gates, code coverage, and PR decoration.
Quality gates	Exit-code based: fail CI on severity threshold via --exit-code and --severity.	Built-in Quality Gates: configurable conditions (coverage, duplications, new bugs) block PRs.
Licensing	Open source (Apache 2.0). No paid tier required for any feature.	Community Edition free; Developer/Enterprise/Data Center editions are paid (branch analysis, advanced security require paid).
Best at	Catching known CVEs in dependencies and base images, hardening IaC, generating SBOMs.	Catching new bugs in code being written: null derefs, injection paths, complexity, duplication.

- **Praveen Kankatala**

Dimension	Trivy	SonarQube
Weak at	Logic bugs in your own source code — it does not parse application code semantics.	Container image scanning, OS package CVEs, K8s misconfig, runtime cluster posture.

When to use which

- Use SonarQube while code is being written and reviewed. It catches bugs in your own source code, enforces quality gates on pull requests, and tracks technical debt over time. Its sweet spot is the developer workflow, IDE through code review.
- Use Trivy once code has become an artefact you ship. It catches known vulnerabilities in the dependencies and base images you pull in, finds misconfigurations in your IaC and Kubernetes manifests, detects leaked secrets, and produces SBOMs. Its sweet spot is the build, registry, and runtime stages.
- Use both together. A mature pipeline typically runs SonarQube on the source code at PR time, then runs Trivy on the resulting container image, IaC files, and SBOM before promotion to staging or production. Neither tool covers the other's ground.

Closing Thoughts

Security tooling works best when it disappears into your workflow. The best outcome from reading this guide is not that you scan more it is that scanning becomes invisible, something that just happens whenever you build, commit, push, or deploy.

Start small. Pick one place a local Dockerfile, a single pipeline, a critical repository and add Trivy there. Make sure the team can read the output and act on it. Once that feels normal, add the next place. Then the next.

Before long, you will have layered scanning at every meaningful point: developer laptop, commit, pull request, build, deployment, and runtime. That is what a real shift-left security posture looks like, and Trivy is one of the easiest tools to get you there.

Happy scanning.

■ **Praveen Kankatala**